Практикум по внедрению версии

План внедрения и передача версии пользователю

За одну практическую пару вы подготовите план обновления, выпустите согласованный комплект файлов и проверите его на локальном стенде. После учебной ошибки выполните откат, повторную установку и оформите передачу результата пользователю.

Связь с предыдущей работой

Руководство оператора теперь становится частью выпуска. Откройте свою предыдущую работу рядом и используйте её как образец точных действий. Для запуска этой практики готовый результат первой пары не обязателен: в новом проекте есть сокращённое исходное руководство. Старую рабочую папку не изменяйте.

Этап самостоятельной работы	Время
Подготовка, план, исходная установка и резерв	20 минут
Новый выпуск, первая попытка и откат	25 минут
Повторная установка, протокол и архив сдачи	15 минут

До практики предусмотрено 25 минут объяснения по презентации и после неё 5 минут обсуждения. Работайте индивидуально. Самопроверку выполняете вы, оценивание проводит преподаватель. Ссылку и срок загрузки в ЭОС объявят отдельно; до этого сохраняйте файлы локально.

Как пользоваться примерами

Страницы 2–15 идут по порядку. Иллюстрации терминала подготовлены по фактическому проверочному запуску; это не снимки рабочего стола Windows. Команды повторены текстом. Не вводите начальный знак $ с картинки. Хеши и число изменённых строк могут отличаться.

Вся работа выполняется внутри deployment-lab. Стенд содержит только учебные файлы, без службы и базы данных. Не переносите команды в папки реальных программ. Продолжайте с сохранённого шага: команды mv и cp не рассчитаны на повтор всего маршрута в уже изменённой папке.

1 Подготовка Windows и рабочей папки

Нужны Windows 10/11 и уже установленный Git for Windows. Скачайте стартовый ZIP, в Проводнике выберите «Извлечь всё» и откройте deployment-lab. Дважды щёлкните 00_OPEN_GIT_BASH.cmd. Резервный способ: контекстное меню этой папки → Open Git Bash here; в Windows 11 может понадобиться «Показать дополнительные параметры».

Все команды вводятся в Git Bash по одной строке. Проверьте pwd: путь должен оканчиваться deployment-lab. Затем выполните команды ниже. Вместо Student Example укажите своё имя. init.sh запускается только один раз.

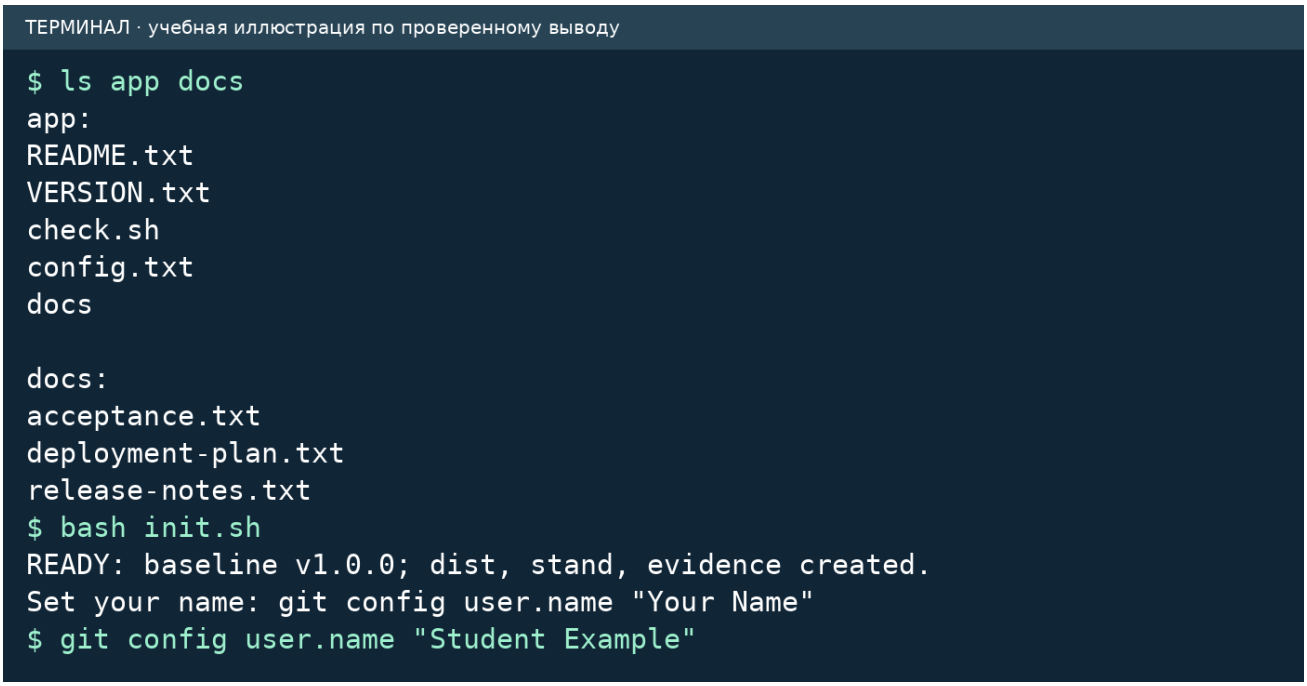


Рисунок 1. Исходные папки, подготовка репозитория и локальное имя автора.

```
ls app docs
bash init.sh
git config user.name "Student Example"
```

READY означает, что подготовка завершена. init.sh создал ветку main, исходный коммит, тег v1.0.0 и служебные каталоги. Сообщение STOP: repository exists означает, что подготовка уже выполнена. Не удаляйте .git и не запускайте всё заново.

Документы редактируются в Блокноте, команды выполняются в Git Bash. Включите отображение расширений в Проводнике и сохраняйте текст как UTF-8. Не создавайте случайно файлы с двойным расширением .txt.txt.

2 Исходная версия и назначение папок

Проверьте исходные значения. VERSION.txt содержит v1.0.0, в config.txt указаны interval=60 и retries=3. Руководство должно относиться к тому же выпуску. check.sh проверяет допустимость настройки и её соответствие README; он не запускает реальный мониторинг.

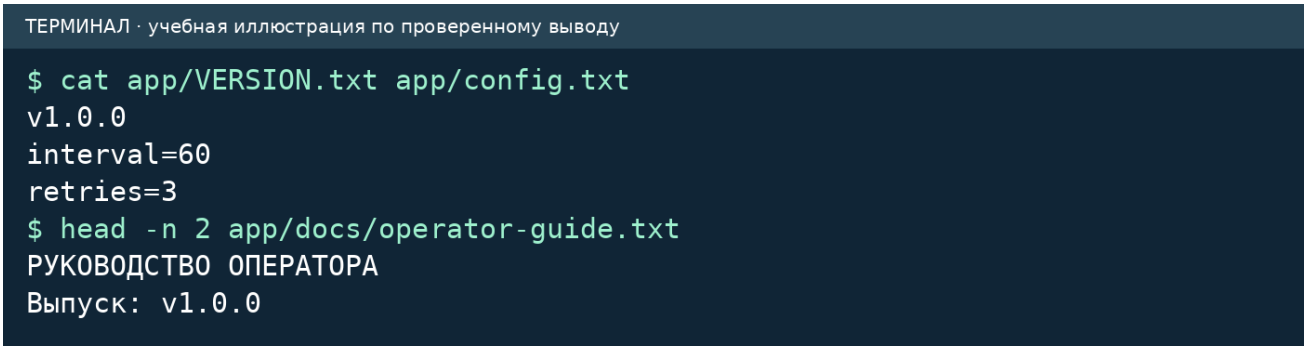


Рисунок 2. Номер исходного выпуска и его настройка.

```
cat app/VERSION.txt app/config.txt
head -n 2 app/docs/operator-guide.txt
```

Путь	Назначение
app	Исходники выпуска; эти файлы сохраняются в Git.
app/docs/operator-guide.txt	Руководство, которое получает пользователь.
docs	План, описание выпуска и протокол внедрения.
dist	Архивы двух выпусков и контрольные суммы.
stand/app	Установленная копия в учебной среде.
evidence	Сообщения и коды фактически выполненных проверок.

app и stand/app — разные каталоги. Изменение исходников не обновляет установленную копию автоматически. Внедрение заключается в передаче сохранённого выпуска в среду и проверке именно этой копии.

Что проверить в руководстве

Найдите четыре раздела, строку «Выпуск» и строку «Ожидается». Сравните их с подходом первой пары. В новой версии потребуется согласованно изменить номер и ожидаемый результат. Остальные объяснения должны оставаться понятными пользователю.

3 План до начала обновления

В Блокноте откройте docs/deployment-plan.txt. Заполните все поля до выполнения установки. Около одной содержательной страницы достаточно. Укажите своё имя, исходную v1.0.0, целевую v1.1.0 и изменение интервала с 60 на 30.

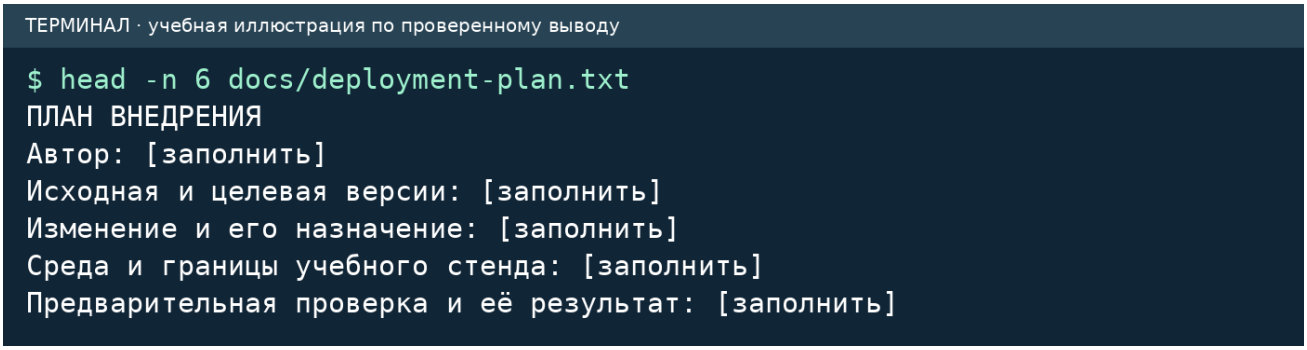


Рисунок 3. Начало шаблона плана; подсказки заменяются собственным текстом.

`head -n 6 docs/deployment-plan.txt`

Используйте последующие страницы, чтобы заранее записать порядок действий. Для каждого существенного шага нужны исходная папка, команда, ожидаемый результат и решение при отказе. В критериях приёмки укажите версию, интервал, согласованное руководство, код 0 и совпадение файлов.

Минимально достаточный фрагмент

«После успешной проверки stand/app создать stand/backup-v1.0.0 командой cp -R и проверить резерв с меткой backup. При отказе новой копии сохранить её в stand/failed-v1.1.0, вернуть резерв в stand/app и проверить v1.0.0 с interval=60». Дополните этот фрагмент точными командами со страниц 6 и 11.

Условие остановки: ненулевой код, неверная версия, несогласованное руководство или несовпадение файлов. Условие повторной попытки: причина отказа установлена, исходный архив цел, новая распаковка прошла проверку. Укажите, что модель не охватывает миграцию базы данных и работающий сервис.

План можно уточнить после самопроверки, но первоначальную последовательность продумайте сейчас. Фактические результаты позже попадут в отдельный протокол; не записывайте заранее, что все проверки пройдены.

4 Архив исходного выпуска

Создайте архив из тега v1.0.0. Последнее слово app ограничивает состав только папкой выпуска. Каталог dist уже создан помощником. Второй командой посмотрите список файлов, ничего не распаковывая.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ git archive --format=tar -o dist/v1.0.0.tar v1.0.0 app
$ tar -tf dist/v1.0.0.tar
app/
app/README.txt
app/VERSION.txt
app/check.sh
app/config.txt
app/docs/
app/docs/operator-guide.txt
```

Рисунок 4. Архив исходного выпуска содержит папку app и руководство.

```
git archive --format=tar -o dist/v1.0.0.tar v1.0.0 app
tar -tf dist/v1.0.0.tar
```

В архиве должны присутствовать check.sh, config.txt, README.txt, VERSION.txt и docs/operator-guide.txt внутри app. Шаблоны из корневого docs и помощники в выпуск не включаются. Архив создаётся из тега, а не из текущих несохранённых файлов.

Почему используется TAR

TAR здесь является упаковкой файлов. Его можно распаковать командой tar из Git Bash, поэтому не нужен отдельный архиватор. Пользовательский результат сдачи позже будет обычным ZIP. Эти два формата выполняют разные задачи в маршруте.

Контрольная точка

Убедитесь, что файл dist/v1.0.0.tar существует и список соответствует рисунку по составу. Порядок имён не принципиален. Если Git не находит тег, проверьте успешное завершение init.sh и текущую папку. Не создавайте второй произвольный исходный тег.

Архив не содержит историю .git. Для восстановления истории при проверке работы будет отдельный Git-bundle. Пользователю для обычного запуска программы история разработки не требуется.

5 Исходная установка и проверенный резерв

Каталог stand после init.sh пуст. Распакуйте архив в него, затем проверьте stand/app. probe.sh сохраняет текст и код в evidence; успех исходной установки — VERSION v1.0.0, PASS с interval=60 и RESULT с code=0.

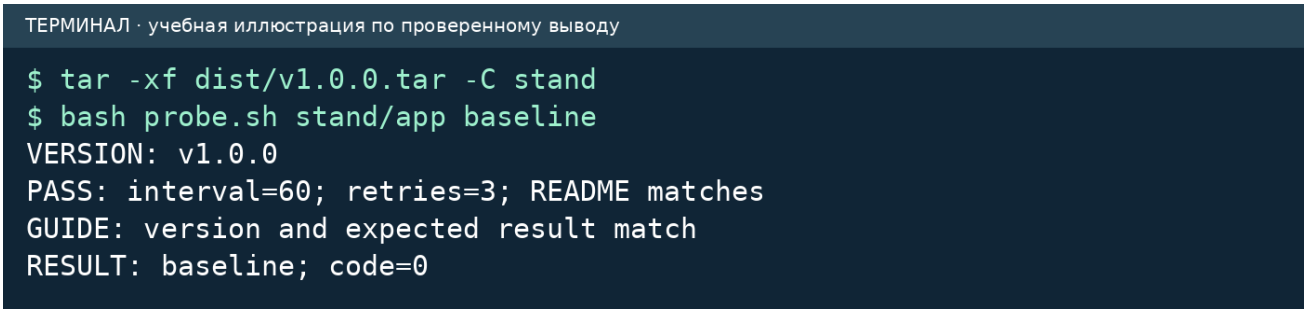


Рисунок 5. Проверка выполняется из установленной копии.

```
tar -xf dist/v1.0.0.tar -C stand
bash probe.sh stand/app baseline
```

Только после успеха создайте резерв. Перед cp каталог stand/backup-v1.0.0 не должен существовать. Команды ниже выполняются один раз; копирование в существующую папку создаст лишнюю вложенность.

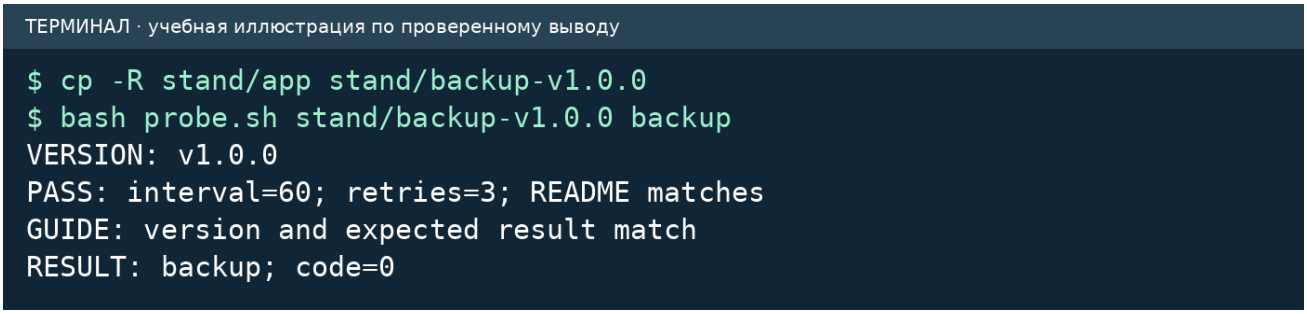


Рисунок 6. Резервная копия также прошла проверку.

```
cp -R stand/app stand/backup-v1.0.0
bash probe.sh stand/backup-v1.0.0 backup
```

Сохранены протоколы baseline и backup. Если любой из них неуспешен, не переходите к изменению версии: прочитайте сообщение, проверьте путь и состав файлов. Работающая резервная копия нужна до первого обновления.

6 Изменение конфигурации и руководства

Через Блокнот измените четыре файла в исходной папке app. В config.txt сохраните ровно две строки: interval=30 и retries=3. В README.txt запишите Monitoring interval: 30 seconds. В VERSION.txt оставьте одну строку v1.1.0.

В app/docs/operator-guide.txt замените строку на «Выпуск: v1.1.0» и строку ожидаемого ответа на «Ожидается: PASS: interval=30; retries=3; README matches». Остальные допустимые значения 30 или 60 описывают возможности проверки и остаются верными. Сохраните все файлы в UTF-8.

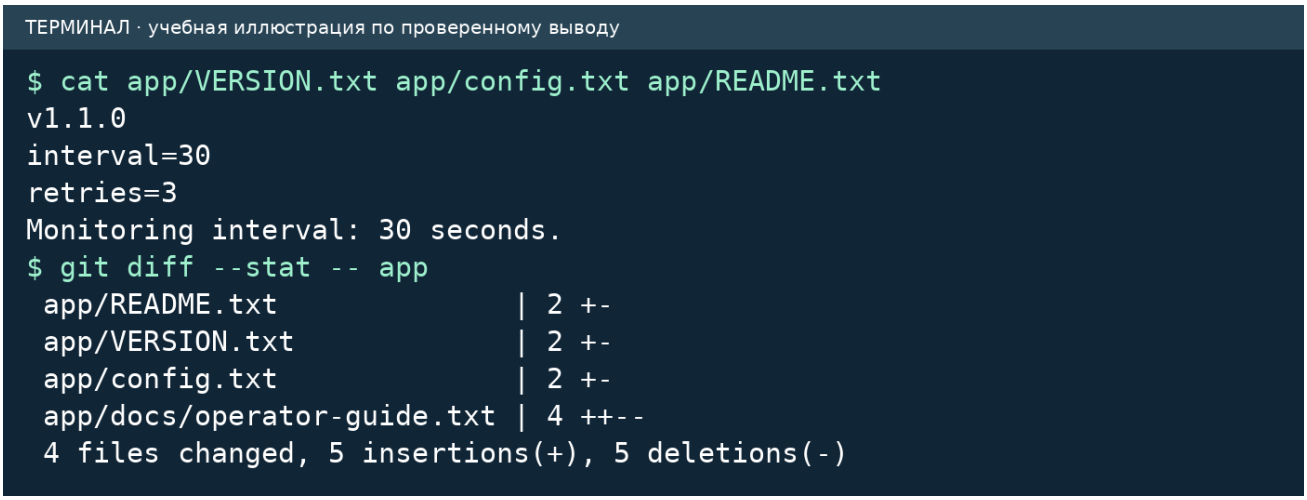


Рисунок 7. Четыре файла исходного выпуска изменены согласованно.

```
cat app/VERSION.txt app/config.txt app/README.txt
git diff --stat -- app
```

Описание выпуска

Заполните docs/release-notes.txt. Укажите изменение интервала, четыре изменённых файла, ожидаемый PASS и код 0. Код check.sh не менялся; приложение остаётся учебной проверкой файлов. Укажите порядок обращения за помощью: сохранить команду и вывод и передать преподавателю при консультации.

Сверьте название выпуска и настройку во всех документах. Не меняйте stand/app: там пока должна оставаться v1.0.0. Подготовленная новая версия и установленная старая версия могут существовать одновременно.

7 Коммит и тег нового выпуска

Перед фиксацией убедитесь, что план и описание выпуска заполнены. Протокол acceptance пока остаётся шаблоном: он будет заполнен по результатам. Добавьте app и два документа, создайте коммит, затем тег v1.1.0.

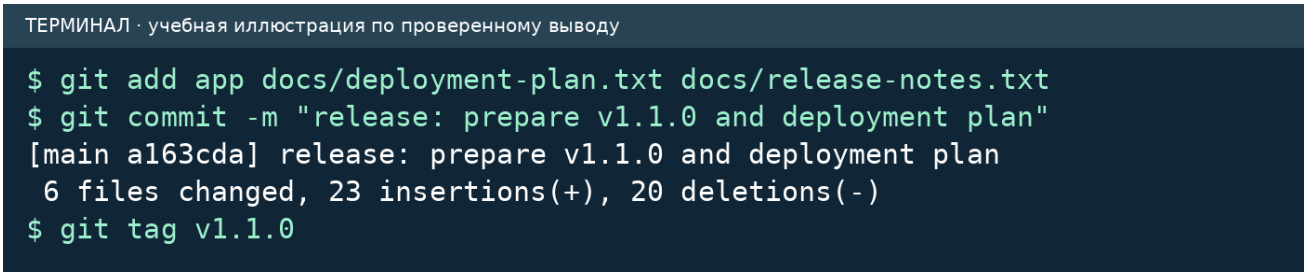


Рисунок 8. Новый выпуск зафиксирован; хеш и число строк у вас могут отличаться.

```
git add app docs/deployment-plan.txt docs/release-notes.txt
git commit -m "release: prepare v1.1.0 and deployment plan"
git tag v1.1.0
```

Тег обозначает состояние файлов, из которого создаётся архив. Коммит нужен до тега. Если забыть сохранить документ в Блокноте, Git не сможет включить несохранённый текст. Команда git status --short помогает проверить оставшиеся изменения.

Если обнаружили ошибку

До создания тега исправьте файлы и сохраните коммит. Если тег уже создан и в самом выпуске есть ошибка, остановитесь и покажите преподавателю git status --short и содержание файлов. Не применяйте принудительное перемещение тега. Для повторного учебного прохождения можно распаковать стартовый архив в новую папку, сохранив прежнюю работу.

Две версии документов

Позднее протокол появится в новом коммите после тега. Это ожидаемо: результат проверки создаётся после выпуска. При сдаче отдельно фиксируется коммит тега v1.1.0 и сохраняется вся история. План и описание выпуска уже доступны в коммите подготовки.

8 Архив новой версии и контрольные суммы

Создайте второй архив из тега v1.1.0. Затем вычислите суммы двух архивов и сохраните их в dist/SHA256SUMS.txt. Выполните проверку по этому файлу; должны появиться две строки OK.

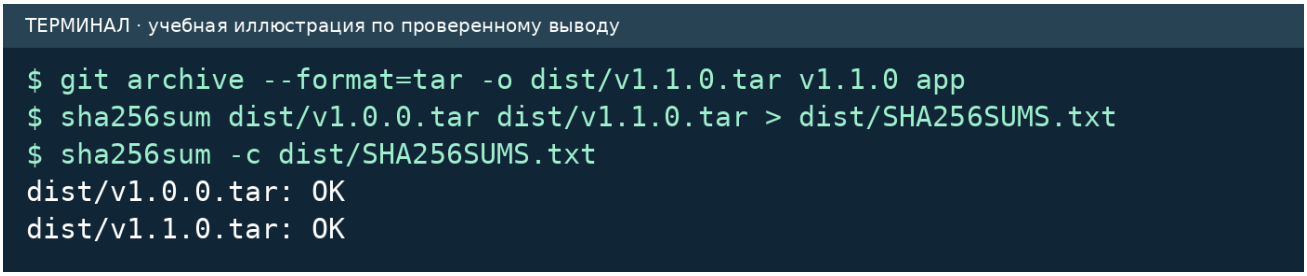


Рисунок 9. Архивы совпадают со своими сохранёнными контрольными суммами.

```
git archive --format=tar -o dist/v1.1.0.tar v1.1.0 app
sha256sum dist/v1.0.0.tar dist/v1.1.0.tar > dist/SHA256SUMS.txt
sha256sum -c dist/SHA256SUMS.txt
```

Все команды выполняются из deployment-lab. В манифест записаны относительные пути dist/v1.0.0.tar и dist/v1.1.0.tar, поэтому проверять его нужно из той же корневой папки. Длинная команда на рисунке может перенестись визуально: вводите её одной строкой.

Что доказывает эта проверка

OK означает совпадение байтов с записанной суммой. Это не проверка работоспособности программы и не подтверждение личности автора. Если файл сумм и архив изменены вместе, такое сравнение не обнаружит подмену. В работе дополнительно проверяются тег, установленная копия и фактический запуск.

Если появилось FAILED

Остановитесь. Проверьте, не был ли архив заново создан или изменён после вычисления сумм. Установите причину прежде, чем пересоздавать манифест. Не скрывайте расхождение переписыванием слова FAILED в протоколе.

9 Первая попытка и учебный отказ

Перед этим шагом stand/app содержит исправную v1.0.0, резерв проверен, stand/before-update ещё не существует. Переместите старую установленную копию, распакуйте новую и специально переименуйте README только внутри stand/app.



Рисунок 10. Ожидаемый отказ из-за отсутствия README.txt в установленной копии.

```
mv stand/app stand/before-update
tar -xf dist/v1.1.0.tar -C stand
mv stand/app/README.txt stand/app/README.saved
bash probe.sh stand/app failed
```

В этом блоке код 1 ожидается. Он означает, что приёмку нужно остановить. Текст FAIL: required file missing сохранён в evidence/failed.txt, а код — в evidence/failed_code.txt. Помощник сам записывает код сразу после проверки.

Причина отказа

Файл не пропал из Git и не повреждён в архиве. Его специально переименовали в stand/app/README.saved. Архив dist/v1.1.0.tar остался исправным. Укажите именно эту причину в протоколе; не называйте её ошибкой кода check.sh.

Не исправляйте ситуацию возвратом одного имени на этом шаге. Задача упражнения — проверить предусмотренный в плане полный возврат к прежнему выпуску. Неудачная копия будет сохранена отдельно для разбора.

10 Возврат к предыдущему выпуску

Сохраните неудачную копию под именем stand/failed-v1.1.0. Затем скопируйте проверенный резерв обратно в stand/app. Папка stand/app после mv должна отсутствовать; именно поэтому cp создаст нужную копию без лишнего вложенного уровня.



Рисунок 11. После возврата снова работают v1.0.0 и interval=60.

```
mv stand/app stand/failed-v1.1.0
cp -R stand/backup-v1.0.0 stand/app
bash probe.sh stand/app rollback
```

Успешный откат подтверждён VERSION: v1.0.0, PASS с interval=60, согласованным руководством и code=0. Эти данные записаны в evidence/rollback.txt и evidence/rollback_code.txt. Если там новая версия или иной код, возврат нельзя считать завершённым.

Что осталось в проекте

app по-прежнему содержит исходники v1.1.0. dist хранит оба архива. stand/app теперь содержит старый установленный выпуск. В stand/failed-v1.1.0 находится неудачная копия новой версии. Резерв backup-v1.0.0 сохранён.

Откат установленной копии не удаляет коммит и тег новой версии. Команды git reset и git revert для этой операции не нужны. Мы меняем состояние учебной среды, а историю разработки сохраняем.

В этой модели нет новых пользовательских данных. Для реальной системы нельзя автоматически считать, что возврат старых файлов восстановит данные или совместимость схемы базы. Это ограничение нужно явно указать в плане и протоколе.

11 Проверка повторной попытки

Создайте новую папку retry и распакуйте в неё исходный архив v1.1.0. В ней ещё не выполнялось учебное переименование README. Проверьте кандидата до переноса на рабочее место.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ mkdir -p stand/retry
$ tar -xf dist/v1.1.0.tar -C stand/retry
$ bash probe.sh stand/retry/app candidate
VERSION: v1.1.0
PASS: interval=30; retries=3; README matches
GUIDE: version and expected result match
RESULT: candidate; code=0
```

Рисунок 12. Чистая распаковка исправного архива успешно проверена.

```
mkdir -p stand/retry
tar -xf dist/v1.1.0.tar -C stand/retry
bash probe.sh stand/retry/app candidate
```

Только при code=0 выполните следующий блок. Сохраните восстановленную старую копию и перенесите проверенного кандидата в stand/app. Имена целевых папок до переноса должны быть свободны.

ТЕРМИНАЛ · учебная иллюстрация по проверенному выводу

```
$ mv stand/app stand/restored-v1.0.0
$ mv stand/retry/app stand/app
$ bash probe.sh stand/app accepted
VERSION: v1.1.0
PASS: interval=30; retries=3; README matches
GUIDE: version and expected result match
RESULT: accepted; code=0
```

Рисунок 13. Финальная установленная копия относится к v1.1.0.

```
mv stand/app stand/restored-v1.0.0
mv stand/retry/app stand/app
bash probe.sh stand/app accepted
```

Если candidate завершился отказом, не выполняйте перенос. Запишите сообщение и выясните причину. В обязательном маршруте исправный исходный архив должен дать PASS с interval=30 и согласованным руководством.

12 Контроль состава и самопроверка

Сравните исходную папку app с окончательно установленной stand/app. diff -qr ничего не печатает при совпадении. Сразу после него введите echo $?; ожидается 0. Если вставить другую команду между ними, будет показан код уже другой операции.

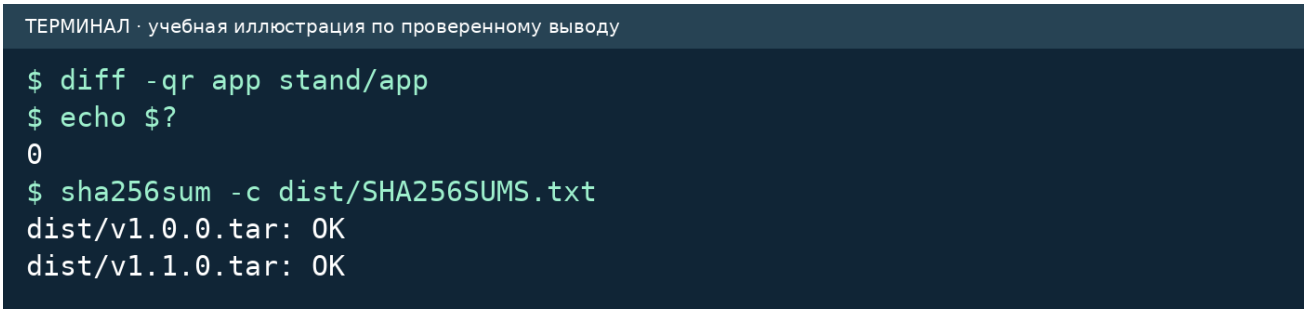


Рисунок 14. Файлы совпадают, обе суммы архивов подтверждены.

```
diff -qr app stand/app
echo $?
sha256sum -c dist/SHA256SUMS.txt
```

Строка Only in или Files ... differ означает расхождение. Не удаляйте файлы наугад: сначала установите, что изменилось. Для данного выпуска ожидается точное совпадение. В реальных установках допустимые локальные отличия должны быть заранее описаны.

Пункт самопроверки	Что подтвердить
План	По тексту понятны порядок обновления, резерв и откат.
Факты	Есть шесть протоколов: baseline, backup, failed, rollback, candidate, accepted.
Руководство	В установленной копии верно указаны версия, команда и ожидаемый результат.
Решение	Принятие связано с критериями; ограничения записаны.

Проверяйте собственную работу. probe.sh сверяет строки версии и ожидаемого ответа, но не оценивает ясность всего руководства. Прочитайте текст из stand/app/docs и проверьте, можно ли выполнить указанные действия без догадок.

13 Протокол и передача пользователю

В Блокноте заполните docs/acceptance.txt по фактическим наблюдениям. Запишите пять успешных кодов 0 и один ожидаемый код 1, причину отказа, решение об откате и результат повторной попытки. Перенесите полный идентификатор коммита выпуска из первой команды ниже.

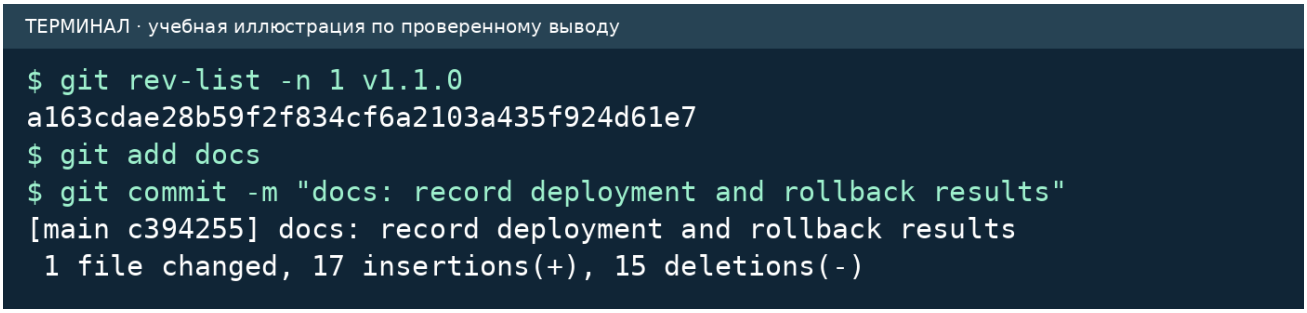


Рисунок 15. Протокол сохранён отдельным коммитом после выпуска.

```
git rev-list -n 1 v1.1.0
git add docs
git commit -m "docs: record deployment and rollback results"
```

До git add замените все подсказки [заполнить] в трёх документах. В протоколе должны быть результаты самопроверки, перечень передаваемых файлов, команда запуска и ограничения. Если обязательный критерий не выполнен, итоговое решение — «не принять» с причиной; не подставляйте успешный результат с картинки.

Пример сообщения пользователю

«Передаётся выпуск v1.1.0 с интервалом 30. Распакуйте архив, откройте Git Bash в папке app и выполните bash check.sh. Ожидается PASS с interval=30 и код 0. Руководство находится в app/docs. При ошибке сохраните команду и вывод и следуйте согласованному порядку обращения за помощью». Дополните описание ограничениями учебной модели.

В протоколе указывается коммит тега v1.1.0, а не новый коммит отчёта. Хеш из рисунка не копируйте: у вас собственная история. Для итогового сравнения важно, чтобы app после тега не изменялся.

14 Сбор и сохранение результата

Выполните pack.sh после коммита документов. Он проверит заполнение шаблонов, коды наблюдений, контрольные суммы, соответствие архивов тегам и установленным файлам. Оценку содержания текста выставляет преподаватель.

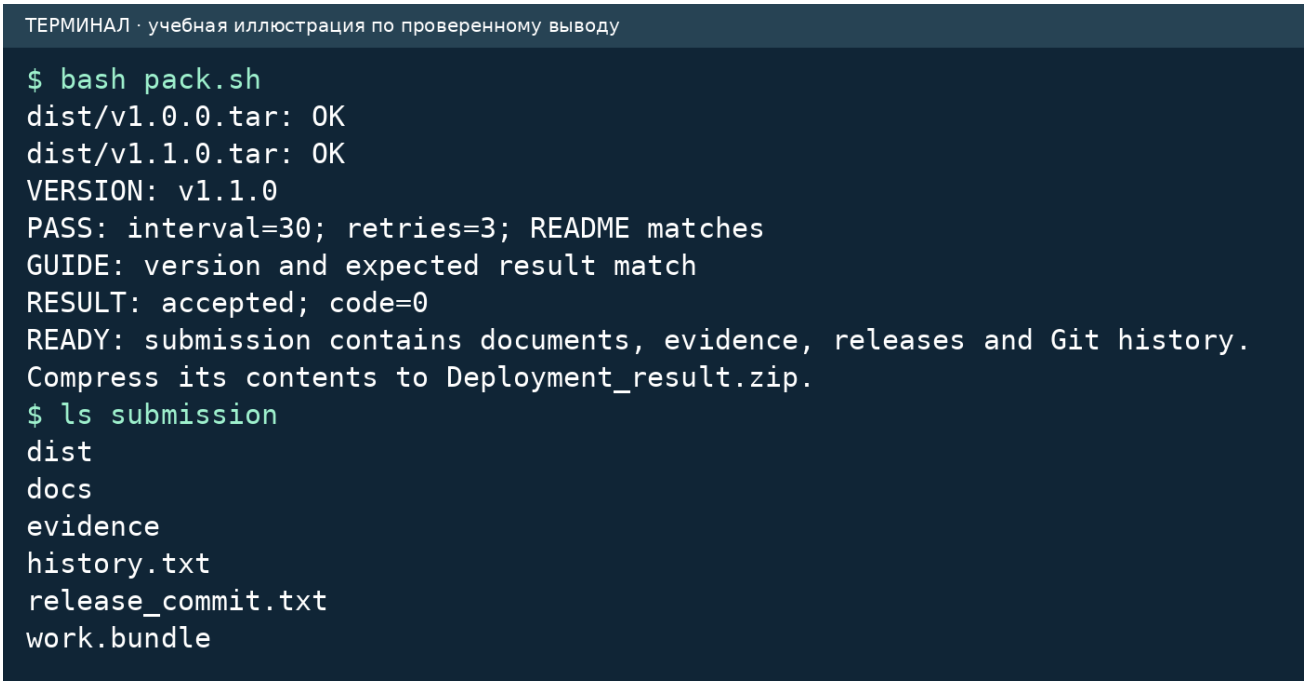


Рисунок 16. Комплект сдачи собран в submission.

```
bash pack.sh
ls submission
```

Откройте submission в Проводнике, выделите содержимое и создайте сжатую ZIP-папку. Назовите архив Deployment_result.zip и перенесите рядом с submission. Внутри нужны docs, evidence, dist, release_commit.txt, history.txt и work.bundle. Сохраните также исходную рабочую папку.

Позже загрузите один ZIP в задание ЭОС по объявленной ссылке. Если требуется отдельная окончательная отправка после загрузки, подтвердите её соответствующей кнопкой. Пока задание не опубликовано, достаточно локально сохранить результат.

Если упаковка остановилась

Complete all three documents: найдите оставшиеся подсказки. Commit the completed documents: сохраните изменения коммитом. Installed files differ: вернитесь к сравнению app и stand/app. Archive checksum mismatch: выясните причину изменения архива. Не меняйте код помощника, чтобы скрыть ошибку.

15 Критерии и продолжение работы

Критерий	Полное выполнение на 2 балла
План внедрения	Есть исходная и целевая версии, пути, предварительная проверка, резерв, критерии приёмки и команды отката.
Согласованный выпуск	Конфигурация, README, версия и руководство согласованы; тег и архив соответствуют сохранённым файлам.
Ошибка и откат	Сохранены ожидаемый отказ и успешный возврат v1.0.0; причина ошибки объяснена.
Приёмка и передача	Есть повторная проверка v1.1.0, сравнение файлов, контрольные суммы, решение и ограничения.
Документация и комплект	План, описание выпуска и протокол заполнены, сохранены в Git; архив сдачи полный.

За каждый критерий: 2 — полностью и подтверждено; 1 — частично или с неточностями; 0 — отсутствует или неверно. Максимум 10 баллов. Самопроверка помогает найти ошибки; итоговую оценку выставляет преподаватель.

Контрольные вопросы

1. Почему наличие резервной папки ещё не доказывает готовность отката?
2. Чем откат stand/app отличается от изменения истории Git?
3. Почему два OK у архивов не доказывают полноту установленной копии?
4. Почему повторная распаковка того же архива устранила учебную ошибку?
5. Как связаны коммит выпуска и более поздний коммит протокола?

Если работа прервалась

Запишите последний завершённый шаг и сохраните папку. При продолжении проверьте pwd и содержимое stand. Не повторяйте весь набор mv и cp: это может создать вложенные копии. При сомнении покажите преподавателю ls stand и точную команду. Старую папку сохраните, даже если начинаете новое прохождение.

Дополнительно: добавьте в план действия при несовпадении SHA256SUMS и при устаревшем руководстве. Объясните, какой этап нужно остановить. Менять готовый выпуск для этого задания не требуется.